

Trabalho 2 - CRUD por RMI

Matheus Berbel Barusso

Repositório completo do trabalho disponível em:
<https://github.com/MatheusBarusso/sistemas-distribuidos>

I. FLUXO DE INICIALIZAÇÃO

- 1) Inicialização do Name Server com `pyro5-ns`: O servidor registra o nome dele e o cliente procura esse nome para encontrar o objeto remoto.
 - 2) Inicialização do servidor com `server.py`:
 - a) Importa o Pyro, a classe `MercadoCRUD` do arquivo `crud.py` e as funções para serialização de `serializador.py` e após isso registra os serializadores com `registrar_serializadores()` para enviar e receber objetos da classe `Produto`.
 - b) Cria um daemon Pyro, localiza Name Server, cria uma instância de `MercadoCRUD`, Passa o daemon para o CRUD (para permitir desligamento remoto quando 'E' é selecionado), registra o objeto CRUD no daemon, registra `Matheus.Barusso` no Name Server e entra em `requestLoop` para aguardar chamadas remotas.
 - 3) Classe remota `MercadoCRUD`: É marcada com `@Pyro5.api.expose` para indicar ao Pyro que os métodos da classe podem ser chamados remotamente. Dentro da classe uma instância do banco de dados é criado e os métodos remotos são definidos, eles apenas repassam a chamada para a classe `DB`.
 - 4) Serialização do objeto `Produto`: Como o Pyro5 não aceita (ou pelo menos não consegui fazer com que aceitasse) classes customizadas as funções no arquivo `serializador.py` fazem conversões `Produto` $\longleftrightarrow$ Dicionário. Arquivo é utilizado tanto no cliente quanto no servidor, permitindo que Produtos sejam tanto enviados quanto recebidos.
 - 5) Inicialização do cliente: Em `cliente.py` os serializadores também são registrados, o Name Server é procurado e um proxy é criado. A partir desse momento o `server` é um proxy remoto, quando o cliente chama funções como `server.inserir(...)` ele chama métodos que rodam o servidor.
- 2) **Read (R):** Cliente lê Código de Barras (Primary-Key)
→ Cliente chama `server.buscar(codbar)` → Servidor chama `DB.buscar(codbar)` → DB faz `SELECT` → Se achou, cria `Produto` → Produto retorna ao cliente → Cliente imprime detalhes do produto.

II. FLUXO CRUD

- 1) **Create (C):** Cliente lê dados → Cliente cria `Produto` → Cliente chama `server.inserir(produto)` → Pyro envia o objeto → `MercadoCRUD.inserir` recebe produto → `DB.inserir` grava no `SQLite` → Servidor retorna resposta.